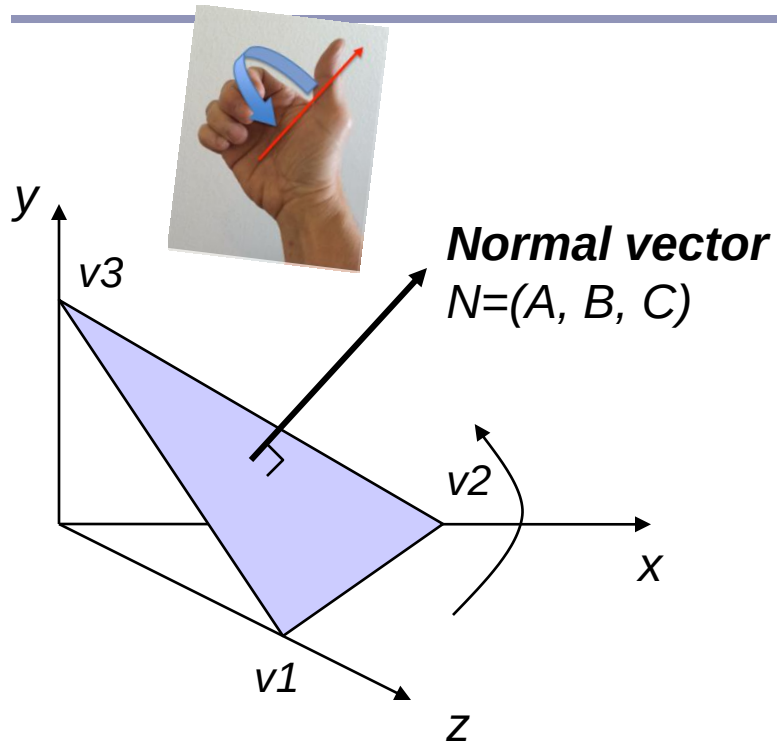


Visible Surface Detection

Front and Back Faces

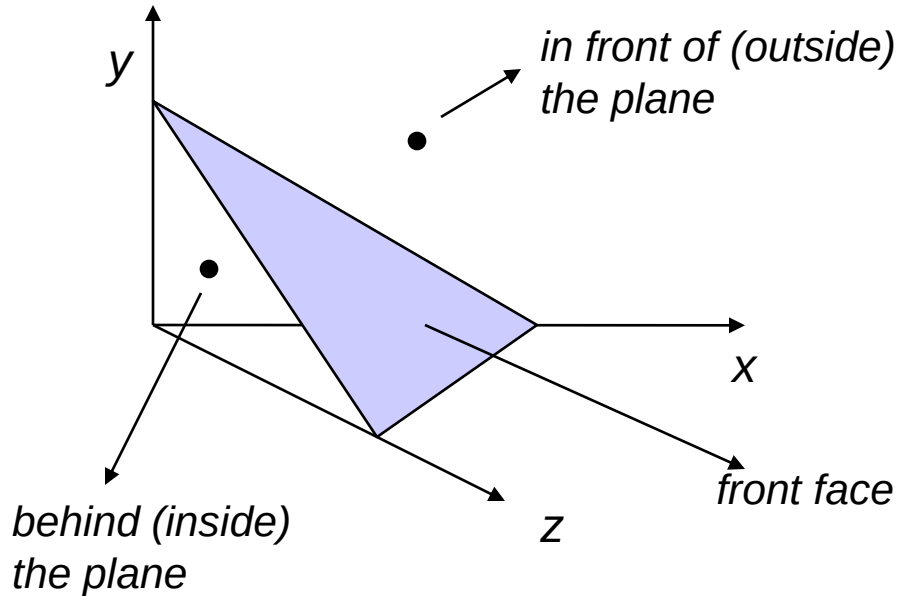


$$A.x + B.y + C.z + D = 0$$

Finding N :

- using Cramer's rule
- using vector cross-product
$$\vec{N} = (\vec{v2}-\vec{v1}) \times (\vec{v3}-\vec{v1})$$
$$\vec{N}.\vec{v1} = -D$$

Front and Back Faces



Any point (x, y, z) is:

on the plane:

$$A.x + B.y + C.z + D = 0$$

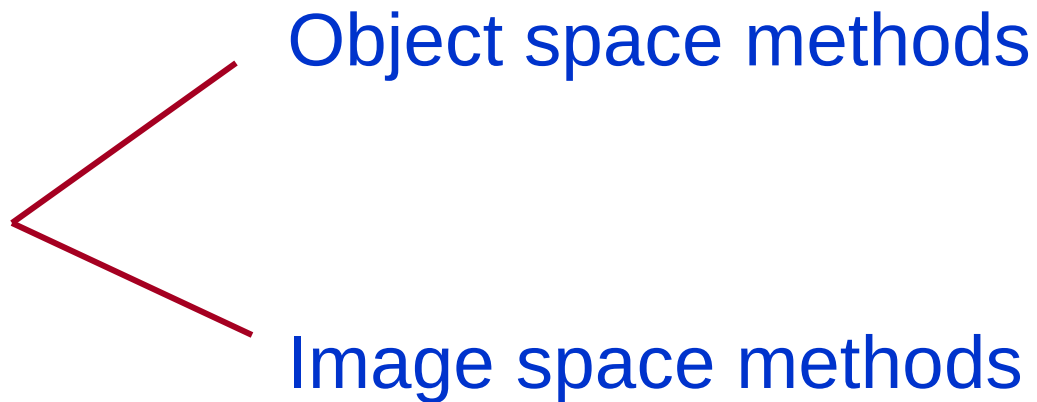
behind the plane:

$$A.x + B.y + C.z + D < 0$$

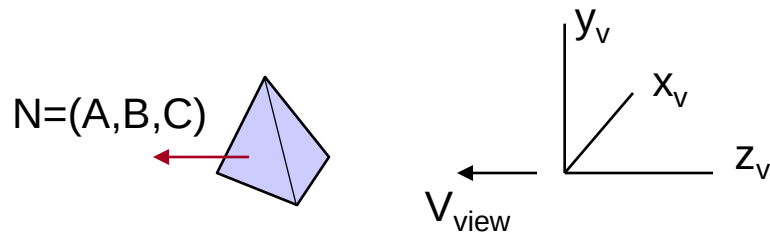
in front of the plane:

$$A.x + B.y + C.z + D > 0$$

Visible Surface Detection



Back-Face Detection

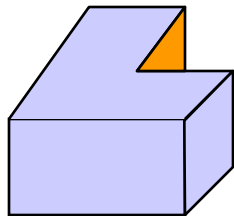


(x,y,z) is behind the polygon if
 $Ax+By+Cz < 0$

or

A polygon is a backface if

$$V_{view} \cdot N > 0$$

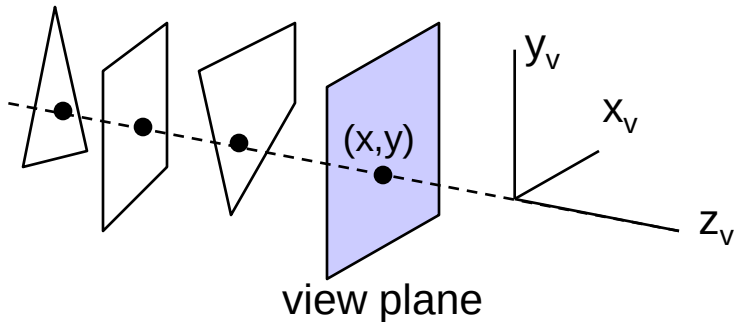


if V_{view} is parallel to z_v axis:

if $C < 0$ then backface

if $C = 0$ then polygon cannot
be seen

Depth-Buffer Method (z-Buffer)



Compares depths of the surfaces and uses the color of the closest one to the view plane

Depth buffer – depth values of surfaces for (x, y)

$$0 \leq \text{depth} \leq 1$$

Frame buffer (refresh buffer) – color value for (x, y)

Object space method

Depth-Buffer Method (z-Buffer)

1. $\text{depthbuffer}(x,y) = 1.0$
 $\text{framebuffer}(x,y) = \text{background color}$
2. Process each polygon one at a time
 - 2.1. For each projected (x,y) pixel position of a polygon,
calculate depth z .
 - 2.2. If $z < \text{depthbuffer}(x,y)$
 compute surface color,
 set $\text{depthbuffer}(x,y) = z$,
 $\text{framebuffer}(x,y) = \text{surfacecolor}(x,y)$

Depth-Buffer Method (z-Buffer)

Calculating Depth:

At (x,y):

$$Ax + By + Cz + D = 0$$

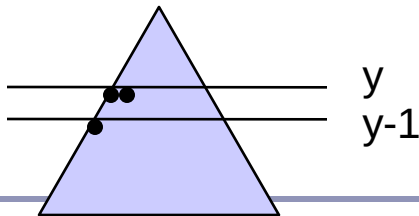
$$z = \frac{-Ax - By - D}{C}$$

For (x+1, y):
$$z' = \frac{-A(x+1) - By - D}{C} = \frac{-Ax - By - D}{C} - \frac{A}{C}$$

$$z' = z - \frac{A}{C}$$

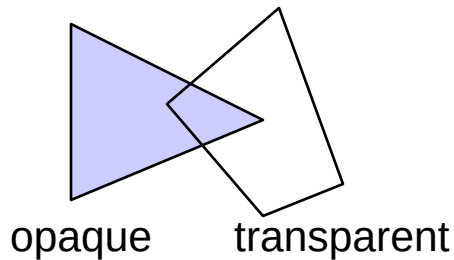
For $y' = y - 1$:
$$x' = x - \frac{1}{m}$$

$$z' = \frac{-A(x - 1/m) - B(y - 1) - D}{C}$$



$$z' = z + \frac{A/m + B}{C}$$

A-Buffer Method



Linked list:

depth	surface info
-------	--------------

Depth: a real number

- ≥ 0 : single surface
- < 0 : multiple surfaces

Surface info: surface data or pointer

Surface data:

- RGB intensity
- opacity
- depth
- percent of area coverage
- surface identifier
- etc.

Scan-Line Method

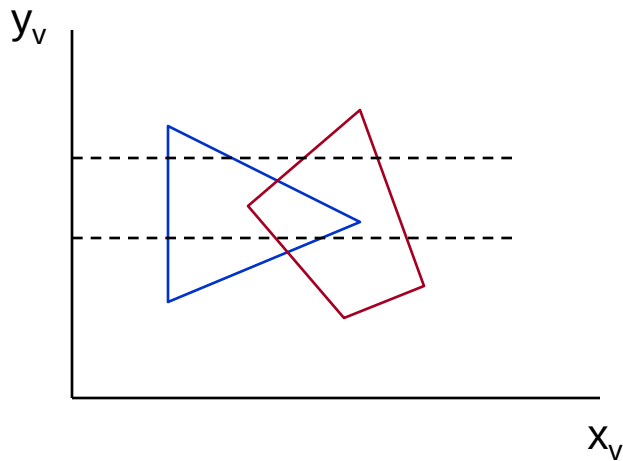


Image space method

For each scan-line, examine all polygon surface projections intersecting that scan line to determine which are visible. Then enter the surface color of that position in the frame buffer.

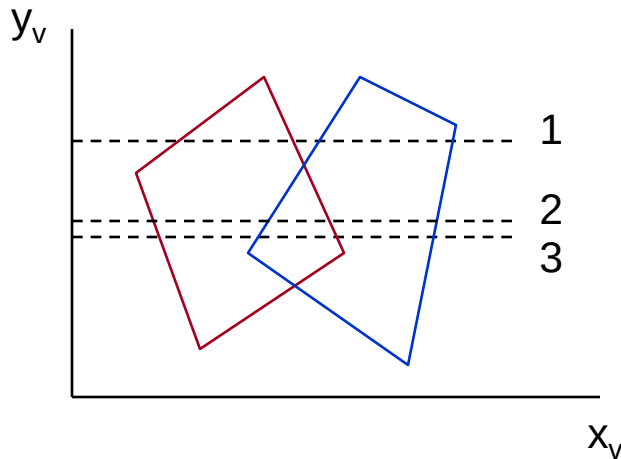
Edge table:

- coordinate endpoints of each line
- inverse slope of each line
- pointers to surface table

Surface table:

- plane coefficients (A,B,C)
- surface material properties
- pointers to edge table

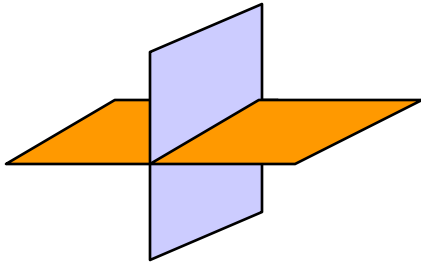
Scan-Line Method



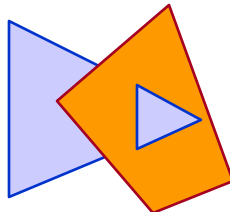
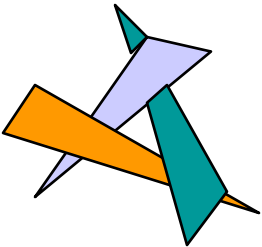
Algorithm:

1. Form an active edge list that contains only the edges that cross the current scan line, sorted in order of increasing x .
2. Define a flag for each surface to indicate whether a position along a scan line is inside or outside the surface.
3. Process pixel positions across each scan line from left to right. Locate visible positions of surfaces along the scan line.

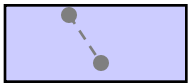
Scan-Line Method



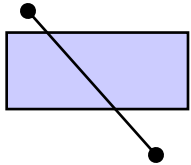
Divide surfaces to eliminate the overlap.



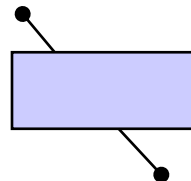
Visible Line Detection (Wireframe visibility)



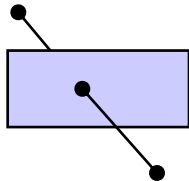
If the projected edge endpoints of a line segment are both within the projected area of a surface, compare the depth of the endpoints to the surface depth at those (x,y) positions.



If both endpoints are behind the surface => **hidden edge**
If both endpoints are in front of the surface => **visible edge**



Otherwise calculate the intersections and the depth value of the intersection point.



- If for both intersection points, edge has greater depth than the surface => **part of the edge is behind the surface**
- If one edge endpoint has greater depth and the other has less depth than the surface => **edge penetrates the surface**

Then, calculate the penetration point

Depth Cueing

Blending objects into the background color with increasing distance from the viewer

$f_{\text{depth}}(d)$ is multiplied by each pixel's color

$$f_{\text{depth}}(d) = (d_{\text{max}} - d) / (d_{\text{max}} - d_{\text{min}})$$

d : distance of a point from the viewing position

d_{min} : 0.0

d_{max} : 1.0

OpenGL

- **glPolygonMode(GL_FRONT_AND_BACK, GL_LINE)**
 - displays both visible and hidden edges
- **glEnable (GL_FOG)**
- **glFogi (GL_FOG_MODE, GL_LINEAR)**
 - generates fog effect. Uses depth cueing.
- **glEnable (GL_CULL_FACE)**
- **glDisable (GL_CULL_FACE)**
- **glCullFace (mode)**
 - removes non-visible faces and renders only visible faces
GL_BACK (default), GL_FRONT (if inside a building),
GL_FRONT_AND_BACK

OpenGL

- **glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB | GLUT_DEPTH)**
 - initializes GLUT, requests a depth buffer and a frame buffer
- **glClear (GL_DEPTH_BUFFER_BIT)**
 - initially depth buffer has the background color. It must be cleared every time before displaying a new frame.
- **glEnable (GL_DEPTH_TEST)**
- **glDisable (GL_DEPTH_TEST)**
 - to activate depth buffer routines
- **glClearDepth (maxDepth)**
 - to set maxDepth to another value (0.0 – 1.0)
- **glDepthRange (nearNormDepth, farNormDepth)**
 - to adjust normalization values (defaults: near- 0.0, far- 1.0)

OpenGL

- **glDepthFunction (testCondition)**
 - to specify a test condition for depth-buffer routines
 - GL_LESS (default), GL_GREATER, GL_EQUAL, GL_NOTEQUAL, GL_LEQUAL, GL_GEQUAL, GL_NEVER, GL_ALWAYS
- **glDepthMask (writeStatus)**
 - to set the status of depth-buffer routines
 - TRUE: read/write
 - FALSE: read only